

OSTIM TECHNICAL UNIVERSITY

Department of Software Engineering

SENG 456

AGENT ORCHESTRATION AND MULTIMODAL SYSTEMS

INDIVIDUAL TERM PROJECT
SUBMISSION FORM

PROJECT TITLE

Autonomous Software Development Team

Full Name	Zain Alabdeen Mohsin Bdaiwi Alkawla
Student ID	220201831
Department / Program	Computer Engineering
Email Address	220201831@ostimtechnik.edu.tr
GitHub / Project URL	github.com/zico20/Dev-Team
Demo / Video URL (if available)	Not applicable – no demo video was recorded for this submission.
Submission Date	19/08/2026

PROJECT SUBMISSION URL

myskillos.com/proje

Upload this form and the project files to the specified address.

Brief Description

This document presents *Autonomous Software Development Team*, a bounded, multi-agent system that takes a natural-language feature request from idea to reviewed, tested, working code. Five role-scoped agents (Product Owner, System Architect, Coder, QA/Tester, Code Reviewer) are orchestrated by Claude Code through a single slash command, with real test execution, JSON-Schema-validated hand-offs at every stage boundary, and a bounded reject/fix loop that escalates to a human instead of retrying forever.

- Explanations below are specific to this project and to the two example features built with it.
- The public source repository, the full audit log, and real pytest output are provided as evidence.
- The project uses a fixed multi-agent workflow (five cooperating Claude Code sub-agents) coordinated by one orchestrating slash command.
- No MCP servers, external APIs, or paid third-party services are used; only Claude Code sub-agents, plain Python, and the standard library.
- No secret keys, passwords, or personal data are included anywhere in the submitted files.

Contents

Brief Description	1
1 Project Summary	2
2 Input, Output, and Usage	3
3 System Design	3
4 Agents, Tools, and Integrations	4
5 Implementation and Execution	6
6 Tests and Results	6
7 Security and Responsible Use	7
8 Conclusion and Future Work	8
9 References and Submission Checklist	9
10 Student Declaration	9

1 Project Summary

Provide a general overview of the project in a few short paragraphs.

The problem the project aims to solve

A realistic software feature request normally passes through several distinct roles – product owner, architect, implementer, tester, and reviewer – each with a different responsibility and a different view of “done.” When a single AI agent is asked to play all of these roles at once, its own approval of its own code is not a trustworthy signal: there is no separation of concerns, no ground-truth verification step, and no defined stopping condition if the agent keeps disagreeing with itself. This project addresses that gap by splitting the process into five least-privilege agents with a real, executable test suite as the ground truth, JSON-Schema validation at every hand-off, and an explicit, bounded retry limit that hands control back to a human instead of looping indefinitely.

The purpose of the project and its target users

The purpose is to demonstrate a small but genuinely trustworthy autonomous development pipeline: one that a developer could hand a feature request to and receive back either working, reviewed code, or a clear, structured explanation of why it could not be produced automatically. The target users are developers or teams who want repeatable, auditable AI-assisted development – with verifiable test results and an audit trail – rather than a single opaque “AI wrote this, trust it” output.

Brief project summary: How does the system work overall, and what does it produce?

The system is implemented as five Claude Code sub-agents plus one orchestrating slash command (`/build-feature`). A *linear build phase* turns a raw feature request into a schema-validated `requirements.md` (Product Owner) and then a schema-validated `architecture.md` design proposal (System Architect); coding does not start until this design exists. The Coder then implements the source code. From this point a *bounded, verified review loop* runs: the real `pytest` suite is executed and converted into a ground-truth `test-results.json`; the QA/Tester (read-only) inspects the code and test output for logic and security issues; the Code Reviewer (read-only) makes the final approve/reject call, and any failing test is an automatic reject regardless of the reviewer’s own opinion. A rejection sends specific feedback back to the Coder for another attempt, up to three attempts; if the third attempt is also rejected, the system builds a structured `escalation.json` for a human instead of retrying a fourth time. Every stage of every attempt is appended to an append-only audit log (`logs/dev-team-audit.jsonl`). The output of a successful run is working, tested source code plus the full paper trail (requirements, architecture, test results, QA report, review verdict) that produced it.

Application domain	Software engineering process automation / multi-agent orchestration
Languages / technologies used	Python 3.12, pytest, jsonschema, Bash, JSON Schema (draft-07), GitHub Actions
Models or services used	Claude Code sub-agents (Claude Sonnet 5), driving five role-scoped agent identities
Current project status	Complete and working end-to-end; validated on two real features and three distinct pipeline scenarios (see Section 6)

2 Input, Output, and Usage

Summarize what the system receives, what it produces, and how it is used.

Primary inputs

A single natural-language feature request, either typed inline as an argument to the `/build-feature` command or provided as a Markdown file (e.g. `feature-request.md`) describing the desired functionality, constraints, and any known edge cases.

Primary outputs

Working, tested source code approved by the Code Reviewer; the supporting documents `requirements.md` and `architecture.md`; the ground-truth `test-results.json`; the `qa-report.json` and `review-result.json`; and, only when the review loop is exhausted after three rejections, a structured `escalation.json` describing exactly what a human needs to look at. Every stage is also appended to `logs/dev-team-audit.jsonl` as a permanent audit trail.

Brief usage scenario: Main steps from the user request to the result

(1) The user writes a feature request. (2) The user runs `/build-feature feature-request.md` inside Claude Code. (3) Product Owner, System Architect, and Coder run in sequence, each output validated against its JSON Schema before the next stage may use it. (4) The real `pytest` suite is executed via `scripts/run_tests.sh`, wrapped in `scripts/run_with_retry.py` for bounded infrastructure retries. (5) QA/Tester and Code Reviewer evaluate the current code and test results. (6a) If approved, the loop ends and the code is ready to use. (6b) If rejected and fewer than three attempts have been made, specific feedback is sent back to the Coder and the loop repeats. (6c) If the third attempt is also rejected, the system builds `escalation.json` and stops instead of retrying again.

A sample input and its corresponding output

Input: “Add a small, self-contained Python utility function that generates a random password guaranteed to satisfy all five rules used by the existing `password_strength.py` checker, using cryptographically secure randomness.”

Output: `password_generator.py`, exposing `generate_password(length=12)`, approved by the Code Reviewer on the first attempt with 33/33 `pytest` tests passing (see Section 6, Trial 4), together with `requirements.md`, `architecture.md`, `test-results.json`, `qa-report.json`, and `review-result.json` for that run.

3 System Design

Describe the architecture only to the extent that it applies to your project.

Why did you choose this approach? Briefly justify the choice of an agent, workflow, or traditional software

A fixed, role-scoped multi-agent workflow was chosen over a single do-everything agent for three reasons. First, *least-privilege tool access* is only meaningful if roles are genuinely separate: the QA/Tester has Read/Grep only and the Code Reviewer has Read only, so neither can silently “fix” the code it is supposed to be gating – a property a single agent cannot have. Second, a *fixed workflow with explicit stage boundaries* makes it possible to validate each hand-off against a JSON Schema and fail fast at exactly the boundary where something went wrong, instead of an error surfacing several steps later in a confusing form. Third, *real test execution* (an actual `pytest` run, not an agent’s self-report) gives a ground-truth signal that a failing test is always an automatic reject, which removes an entire class of the “the agent approved its own mistake” failure mode that a single-agent design is exposed to.

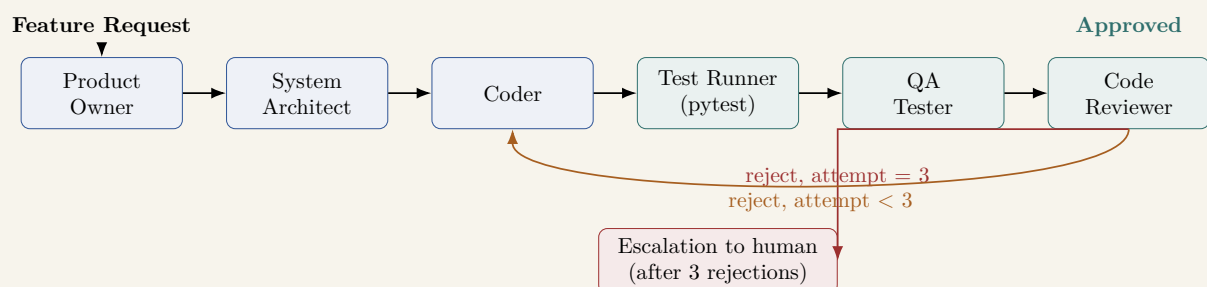
The system’s main components and their responsibilities

Product Owner (Read/Write, own document only) turns the raw request into structured `requirements.md`. **System Architect** (Read/Write, own document only) turns requirements into a `architecture.md` design proposal – a plan for approval, not code. **Coder** (Read/Write, source files only, never test files) implements the design and, on rejection, fixes only what was flagged. **QA/Tester** (Read/Grep only, cannot modify anything) searches the code for logic and security issues and reports them. **Code Reviewer** (Read only) makes the final approve/reject call using QA’s report and the ground-truth test results. Supporting these five agents: `scripts/` (schema validation, the real test runner, bounded infra-retry, escalation building, and audit logging) and `schemas/` (one JSON Schema per agent output).

The main data or task flow between components

`feature-request.md` → `requirements.md` (schema-validated) → `architecture.md` (schema-validated) → Coder writes source → `scripts/run_with_retry.py` wraps the real `pytest` run → `test-results.json` (ground truth) → QA report → Code Reviewer verdict (approved / rejected, with a confidence level) → on rejection with attempts remaining, feedback loops back to the Coder; on the third rejection, `scripts/build_escalation.py` produces `escalation.json` instead. Every stage of every attempt is appended to `logs/dev-team-audit.jsonl`. See the diagram below.

Pipeline architecture diagram:



4 Agents, Tools, and Integrations

If your project does not use agents or external tools, you may write “not used” in the relevant rows.

Component / agent / tool	Brief responsibility	Information received or produced
Product Owner	Converts the raw feature request into unambiguous, structured requirements	Reads the raw request; writes schema-validated requirements.md
System Architect	Designs the system architecture as a proposal for approval, not code	Reads requirements.md ; writes schema-validated architecture.md
Coder	Implements the approved design; on rejection, fixes exactly what was flagged	Reads requirements/architecture (or latest rejection feedback); writes only source files, never tests
QA/Tester	Searches the current code and test results for logic and security issues (read-only)	Reads current source and test-results.json ; returns a structured findings report
Code Reviewer	Makes the final approve/reject decision (read-only, cannot fix anything itself)	Reads QA's report, test results, and code; returns verdict, confidence, and rejection feedback

APIs, data sources, MCP, Skills, models, or other integrations used (if any)

Not used. The project deliberately relies only on Claude Code's built-in sub-agent and slash-command mechanism, plain Python 3, and the Python standard library – no external APIs, no MCP servers, and no third-party services or paid models beyond the Claude models already driving the agents.

Important decisions, permission boundaries, or operations requiring human approval (if any)

Tool access is least-privilege and fixed per role (table in **CLAUDE.md**): Product Owner and System Architect may Read/Write only their own document; the Coder may Read/Write source files only and is explicitly forbidden from touching test files; QA/Tester has Read/Grep only and cannot modify anything; the Code Reviewer has Read only and cannot fix code, only describe what must change. Any failing test, or any high-severity security finding from QA, is an automatic reject with no judgment call. The review loop is bounded at three rejected attempts, after which the system escalates to a human instead of retrying again. Pushing the finished project to the public GitHub repository was an explicit, separate action confirmed with the project owner before it was carried out.

5 Implementation and Execution

Source code URL	github.com/zico20/Dev-Team
Main execution command	<code>/build-feature feature-request.md</code> (run inside Claude Code, in the project directory)
Required core dependencies	Python 3.10+, <code>pytest>=7.0</code> , <code>jsonschema>=4.0</code> (see <code>requirements-dev.txt</code>); a POSIX shell (<code>bash</code>) to run <code>scripts/run_tests.sh</code>
Important files or folders	<code>CLAUDE.md</code> (project-wide rules); <code>.claude/agents/*.md</code> (five agent identities); <code>.claude/commands/build-feature.md</code> (orchestration); <code>schemas/*.json</code> ; <code>scripts/*.py</code> , <code>scripts/run_tests.sh</code> ; <code>logs/dev-team-audit.jsonl</code>
Execution environment	Developed and tested on Windows 11 (PowerShell + Git Bash) via Claude Code; a GitHub Actions workflow (<code>.github/workflows/dev-team-pipeline.yml</code>) runs the same pipeline headlessly on Linux for <code>feature/**</code> branches

Brief installation and usage steps for running the project

(1) Clone the repository and install the Python dependencies: `pip install -r requirements-dev.txt`. (2) Write the desired feature in a Markdown file, e.g. `feature-request.md`. (3) Inside Claude Code, in the project directory, run `/build-feature feature-request.md`. (4) Watch the linear build phase (Product Owner → Architect → Coder) run, followed by the bounded review loop (Test Runner → QA → Reviewer, up to three attempts). (5) Inspect `review-result.json` for the verdict, or `escalation.json` if the loop was exhausted. (6) Optionally push a `feature/**` branch to trigger the same pipeline headlessly in GitHub Actions.

A significant technical problem encountered and the solution applied

The project's ground-truth test-execution script, `scripts/run_tests.sh`, is a Bash script invoked via a shebang line – which works transparently on the Linux CI runner but not on native Windows, where a shebang cannot be executed directly. Beyond that, the script originally forwarded `pytest`'s raw exit code to the caller; since `pytest` exits with code 1 whenever any test simply *fails* (a completely normal outcome, not a tooling crash), the bounded infra-retry wrapper `scripts/run_with_retry.py` was misclassifying ordinary failing-test results as a transient infrastructure error, retrying them needlessly, and ultimately routing them toward an infrastructure escalation instead of letting the Code Reviewer make the intended automatic-reject decision. Both issues were found only by actually executing the pipeline end-to-end, not by reading the code. The fix was two-fold: `run_with_retry.py` now transparently prepends `bash` for any `.sh` command on Windows, and `run_tests.sh` now treats `pytest` exit codes 0 and 1 as a successful run of the *test runner* itself (only codes greater than 1 – usage errors, internal errors, no tests collected – are treated as real infrastructure failures), so a failing test now correctly reaches the review loop as an ordinary, expected rejection instead of being swallowed by the infra-retry path.

6 Tests and Results

Describe the basic checks performed to demonstrate that the project works.

	Trial / test	Result obtained	Status
1	End-to-end pipeline run, Feature 1: <code>password_strength.py</code> (password strength checker)	Approved by Code Reviewer on the first attempt; 8/8 <code>pytest</code> tests passed; QA/Tester found no logic or security issues	Pass
2	Deliberate single-bug injection to exercise the reject/fix loop (an off-by-one rating threshold)	Rejected on attempt 1 with specific, line-level feedback; Coder fixed exactly that issue on attempt 2; 8/8 tests passed; approved on attempt 2	Pass
3	Deliberate three-independent-bug scenario to exercise the escalation path (length threshold, inverted special-character check, comparison-operator bug – one fresh bug per round)	Three consecutive real rejections, each caused by a genuine failing <code>pytest</code> test and confirmed by QA/Tester; the loop stopped after attempt 3 and produced a schema-valid <code>escalation.json</code> instead of attempting a fourth time	Pass
4	End-to-end pipeline run, Feature 2: <code>password_generator.py</code> (cryptographically secure random password generator)	Approved by Code Reviewer on the first attempt; 33/33 <code>pytest</code> tests passed (8 existing + 25 new); QA/Tester specifically verified that only the <code>secrets</code> module (never <code>random</code>) is used for randomness	Pass

Overall result: To what extent did the project achieve its objective?

The project fully achieves its stated objective. Across four independent, real (not simulated) runs it demonstrated: a clean approval on the first attempt for two different features; a genuine reject-then-fix cycle in which the Coder repaired exactly what was flagged and nothing else; and a genuine three-strikes escalation in which the system correctly refused to attempt a fourth fix and instead produced a structured, schema-valid hand-off for a human. Every test result quoted above comes from an actual `pytest` run converted into `test-results.json`, not from an agent's self-report, and the full sequence of every attempt is preserved in `logs/dev-team-audit.jsonl`.

Known limitations or incomplete aspects

All validation in this submission was run manually within a single Claude Code session; the GitHub Actions headless path (`.github/workflows/dev-team-pipeline.yml`) exists and mirrors the same logic but was not separately exercised for this submission. JSON-Schema validation catches structural errors in agent output (missing fields, wrong types) but cannot catch a semantically wrong-but-well-formed answer. Both example features are small, single-file, pure-function utilities; the pipeline has not yet been exercised on a multi-file feature or one that touches a user interface. Finally, the two Windows-specific script bugs described in Section 5 were found only by actually running the pipeline on Windows, which suggests the Linux/CI code path is comparatively more battle-tested than the local Windows path used for this submission.

7 Security and Responsible Use

Evaluate only the items relevant to your project.

OK Check

- ✓ Secret keys and passwords were not included in the source code.
- ✓ User data was processed only to the extent necessary.
- ✓ Risky or irreversible operations were restricted.
- ✓ Sources and third-party content used were identified.

Project-specific security, ethics, privacy, or copyright concerns and the measures taken

No end-user personal data is processed by this project at all – its two example features operate only on password strings supplied directly by a caller and never log, print, or persist them. Because the example domain is password handling, randomness quality was treated as a first-class concern: `password_generator.py` draws exclusively from Python’s `secrets` module (never the non-cryptographic `random` module), and this was explicitly verified by the QA/Tester agent rather than merely assumed. Risky or irreversible operations were kept restricted by design: each agent’s file-system access is scoped to the minimum its role requires (Section 4), destructive git operations were never used, and the one action with external, hard-to-reverse effect in this project – pushing the repository to the public GitHub remote – was explicitly confirmed with the project owner before being carried out rather than performed automatically. No third-party code, datasets, or copyrighted material are included in the submission.

8 Conclusion and Future Work

The most important results and learning outcomes obtained from the project

The most important result is that the bounded, schema-gated, multi-agent review loop was not just designed but actually built and exercised end-to-end, including its least-common paths: a genuine reject-and-fix cycle and a genuine three-strikes escalation, both driven by real agents and a real test suite rather than scripted or simulated for the demonstration. A second, unplanned but valuable outcome was that running the system for real – rather than only reading or reasoning about the code – surfaced two real defects in the orchestration scripts themselves (Section 5), reinforcing that execution-based verification catches classes of bugs that code review alone does not.

Improvements that could be made with more time

With more time, the next steps would be: exercising the GitHub Actions headless path for this same submission rather than only the local run; extending the pipeline to a multi-file feature and to one with a user-facing interface, to test whether the current agent boundaries still hold at larger scope; adding an automated regression check for the two Windows-specific script fixes so they cannot silently regress; and adding lightweight per-stage cost/token tracking to the audit log so the loop’s efficiency, not just its correctness, can be measured over many runs.

Core course concepts applied in the project

Multi-agent orchestration with clearly separated roles; least-privilege permission boundaries enforced per agent; structured, schema-validated inter-agent communication with fail-fast validation at every hand-off; bounded autonomous loops with an explicit stopping condition and human escalation; the distinction between a normal rejection and an infrastructure failure, with separate, appropriately-sized retry policies for each; graceful degradation when an agent’s own output cannot be trusted; and observability via a complete, append-only audit log.

9 References and Submission Checklist

Documents, articles, datasets, models, open-source projects, or other resources used

Claude Code documentation (sub-agents and slash commands); Python `secrets` module documentation (standard library); the JSON Schema specification, draft-07; the `pytest` documentation. No datasets, pretrained models, or third-party open-source code were used beyond the Python standard library and the Claude models already driving the agents.

OK	Submission item
----	-----------------

- | | |
|---|---|
| ✓ | A completed PDF version of this form is included. |
| ✓ | The source code or project files are accessible. |
| ✓ | Installation and execution instructions are included. |
| ✓ | A test result or sample usage result is included. |
| ✓ | No secret keys, passwords, or unauthorized personal data are included. |
| | Demo, video, or supplementary file links work, if provided – not applicable (no demo video was provided for this submission). |

10 Student Declaration

I declare that I prepared this project individually; identified the sources, third-party code, and artificial intelligence assistance I used; and ensured that the submission files contain no secret keys or unauthorized personal data.

Full Name	Zain Alabdeen Mohsin Bdaiwi Almawla
Student ID	220201831
Date	19/08/2026
Signature	_____